TCTools-Build_Web 使用手册

写给从未写过代码的人

版本 **v0.1.0-rc1** · Windows · macOS · Linux

关于本手册中的图片。 本手册中插入了截图占位符，而不是成品截图。真正的图片随 **每个发行版本**一同发布，这样图片永远对应一个已经发布的构建，而不是一个随时变化 的代码库。图片的位置是：

> `docs_Documentation_文档/images_Images_图片/`

下文的每一个占位符都写明了它将来对应的文件名。当你拿到的文档已经包含这些图片时， 它们会出现在完全相同的位置、使用完全相同的相对路径，正文一个字都不用改。

开始之前想说一句。 你不需要先弄懂 HTML、CSS 或 JavaScript 是什么。这几个词 后面都会遇到，读完这本手册，它们就会变成普通的日常词汇。以下所有内容，都是为 “第一次打开这个软件的人”写的。

---

1. 这是什么，能做出什么

TCTools-Build_Web（简称 **TC-BuildWeb**）是一款不用写代码就能制作网页的桌面软件。

整个工作方式只有三步：

1. **把元素拖到页面上。** 标题、图片、按钮，甚至一整个导航栏 —— 从左侧面板里 挑一个，拖到页面上放手即可。
2. **把积木拼在一起，就有行为。** 想让按钮真的能做事？把彩色积木按句子的读法接 起来：*当这个按钮被点击时，显示一条提示*。
3. **导出属于你自己的普通文件。** 得到的就是普通的 `.html`、`.css` 和 `.js`。 不需要账号，不需要订阅，目标端也没有任何专有格式。你可以把文件放到任何网站 空间、发给别人，或者存在 U 盘里。

你可以做出来的东西

你想做的东西	在这里能不能做
个人主页或简历	可以 —— 标题、正文、图片、链接
小型企业官网	可以 —— 导航栏、主视觉、价格表、页脚
图片画廊	可以 —— 直接用"现成版式"里的画廊
联系表单	可以 —— 表单版式，另配一个可选的起始服务器来接收
读取实时数据的页面	可以 —— 用网络请求积木调用接口，并把结果显示在页面上
多页站点	可以 —— 页面数量不限
完全交给 AI 助手搭建的页面	可以 —— TI-UseAgent，每一次改动都需要你确认

它不做什么

它不是图片编辑器，不是电子表格，也不负责**托管**网站。它负责生成文件；把文件放到 网上是另一个很简单的步骤，见第 11 节。

---

2. 安装与启动

运行要求

项目	要求
操作系统	Windows 10（1709 及以上）或 Windows 11；macOS（Apple 芯片或 Intel）；Debian / Ubuntu Linux（x86_64 或 aarch64）
硬盘空间	连同运行库一起安装后，一般几百 MB
显卡与显示	任意能正常显示的台式机、笔记本或虚拟机
网络	只有使用 AI 助手、或页面需要调用接口时才需要。设计、积木、预览和导出都可以离线完成

安装

Windows。 双击安装包（`.exe`）按提示安装即可。如果拿到的是免安装压缩包，那就 不用安装：解压后打开文件夹，双击 `TCTools-Build_Web.exe`。请把整个文件夹一起保留， 程序需要旁边的那些 DLL。

macOS。 打开 `.dmg`，把 **TCTools-Build_Web** 拖进「应用程序」文件夹。第一次 打开时系统可能会询问是否确认：在图标上点右键选择「打开」，确认一次即可；之后正常 双击就能启动。

Linux。 用系统的软件安装器安装 `.deb`，或者直接运行 AppImage：

```
chmod +x TCTools-Build_Web-*.AppImage
./TCTools-Build_Web-*.AppImage
```

如果你想从源码自行构建，完整步骤见仓库的 README.md。

第一次启动

像启动其他程序一样启动它。你会看到一个欢迎界面，上面有三条路：

- **新建工程** —— 从一张空白页面开始。
- **打开工程** —— 打开已有的 `.tcbw` 文件。
- **最近使用的工程** —— 回到上次做过的项目。

欢迎界面 / Welcome screen

`../images_Images_图片/13-welcome-screen.png`

Screenshot placeholder · 截图占位符

如果上一次是异常退出的，欢迎界面还会提供**恢复未保存的内容**。软件在工作过程中会 自动保存，并留有一份崩溃恢复记录；选择"恢复"就会回到它记录下的最后状态。选择 "放弃"只是丢掉这份恢复记录，你真正保存过的文件不受任何影响。

检查环境是否正常：`--doctor`

如果觉得哪里不对劲 —— 某个功能不见了、弹窗很奇怪、或者换了台不熟悉的电脑 —— 可以 运行内置的诊断命令。打开终端（Windows 用 PowerShell，macOS 用「终端」，Linux 用你 的 shell），执行：

```
# Windows
.\TCTools-Build_Web.exe --doctor
```

```
# macOS 与 Linux
./TCTools-Build_Web --doctor
```

它会打印一份简短的报告然后退出。下面是这份报告在开发参考机上的样子（你的版本号、 路径和数量都会不同）：

```
TCTools-Build_Web 0.1.0-rc1 (rc1)
Qt runtime      : 6.11.1
Qt build        : 6.11.1
Platform        : windows / Windows 11
Architecture    : x86_64
Kernel          : winnt 10.0.26100
Locale          : en_US
Settings file   : C:\Users\you\AppData\Roaming\turtlelnc\TCTools-Build_Web.ini
Secret backend  : Windows DPAPI (OS-backed)
Vendors         : <本构建中的 AI 厂商预设数量>
Models          : <已知模型标识的数量>
HTML elements   : <元素面板中的条目数量>
Block types     : <积木工具箱中的积木种类数量>
WebEngine       : not available (external browser preview)
```

怎么读这份报告：

- **Secret backend（凭据后端）** 说明你的 AI 凭据是怎么保护的。显示 *OS-backed* 是最强的情况；显示 *obfuscated* 表示文件只是被打乱了、并没有真正加密 —— 这正是 Linux 上目前的实际情况，软件会照实说明，而不是含糊过去。
- **WebEngine** 说明有没有内置浏览器内核。默认构建是没有的，因此预览的工作方式 见第 4 节与第 11 节。
- **Vendors / Models / HTML elements / Block types** 都是数量。如果某一项是 0，说明 对应的部分没有加载成功，这时重新安装是合适的第一步。

如果要把问题反馈给别人，把这份报告整段贴上就行 —— 它替你回答了一大半对方本来要问 的问题。

---

3. 十分钟做出第一个页面

我们现在真的做一个页面，从空窗口做到成品文件。跟着做就好，做不坏。

第 1 步 —— 新建工程

按 Ctrl+N（macOS 上是 Cmd+N），或者在欢迎界面点「新建工程」。窗口中间会 出现一个页面。软件提示时给工程起个名字，也可以先不管，之后用 Ctrl+Shift+S 另存改名。

此时你处在**设计**模式。窗口中央就是画布：一张空白页面，中间有一块虚线区域，写着 *把元素拖到这里开始*。

第 2 步 —— 拖一个标题进来

看左侧面板，它叫**元素**，把所有能放到页面上的东西分了类：布局、文本、媒体、表单、 数据、导航、高级，另外还有一个**现成版式**分组，里面是已经做好的整块内容，比如导航 栏、主视觉、页脚、价格表、画廊、联系表单和登录表单。

展开**文本**，找到**一级标题**，拖到虚线区域里松手。

画布上出现一个大标题。它已经被选中了 —— 有明显的选中框，右侧面板也正在显示它的 设置。

设计模式 / Design mode

`../images_Images_图片/01-design-mode.png`

Screenshot placeholder · 截图占位符

第 3 步 —— 改文字

标题选中状态下，看右侧的**属性**面板。它有三个标签页：**内容**、**样式**、**属性**。

- **内容** 决定写什么字。
- **样式** 决定长什么样。
- **属性** 是更底层的细节，平时基本用不到。

现在正在**内容**页。在文本框里输入你自己的标题，比如**晨曦面包房**。画布会跟着 你打字实时变化。这里没有“保存”按钮：你的每一步都会被记下来，**Ctrl+Z** 可以撤销 任何操作。

属性面板标签 / Inspector tabs

`../images_Images_图片/03-inspector-tabs.png`

Screenshot placeholder · 截图占位符

第 4 步 —— 改颜色

切到**样式**页，你会看到：

- **快捷样式** —— 一排一键预设。随便点一个试试，不喜欢就按 Ctrl+Z。
- **文字** —— 字体、字号、字重、对齐、行高。
- **颜色** —— 文字颜色；**背景** —— 它背后的颜色。
- **间距、圆角、阴影** —— 元素周围留多少空、角有多圆、要不要投影。

点**颜色**旁边的色块挑一个颜色，标题会立刻跟着变。

第 5 步 —— 加一个按钮

回到**元素**面板，展开**表单**，把**按钮**拖到画布上，放在标题下面。选中它，进入 **内容**页，把按钮文字改成**打个招呼**。

如果按钮落错了位置，在画布上直接拖动它就能移动。拖动时按住 Shift，移动会保持 水平或垂直。

第 6 步 —— 用积木让按钮真的能做点什么

现在到了有意思的部分。按 Ctrl+2 切换到**积木**模式。

积木模式左边是工具箱，中间是脚本区。积木按类别用颜色区分：事件、界面、文本、控制、 逻辑、数学、变量、存储、网络请求、后端、数据、人工智能、MCP 工具、定时、控制台。 后面几类属于**高级**内容，默认是收起来的，打开“显示高级积木”才会出现。

每个脚本都从一个**帽子块**开始 —— 顶部是圆弧、没有凹槽的那种积木，意思是 *当某件事发生时*。从**事件**里把**当按钮被点击**拖到脚本区。

再展开**界面**，把**显示提示**拖到它下面。凹槽对齐时积木会自动吸附：把新积木悬停 在帽子块正下方，听到"咔"地合上就对了。

你的脚本现在读起来是这样：

```
当按钮被点击
    显示提示  "你好！"
```

在提示积木的输入框里点一下，换成你自己的话。这就是一个程序。它也**真的**会被编译 成 JavaScript —— 不是比喻。

积木编辑器 / Blocks editor

`../images_Images_图片/04-blocks-editor.png`

Screenshot placeholder · 截图占位符

右侧面板已经在显示你刚拼出来的**生成的 JavaScript**，上面还有**错误**和**警告** 两个徽标。如果徽标是红的，说明有积木没填完整 —— 通常是某个输入框还空着。

第 7 步 —— 在自己的浏览器里预览

按 Ctrl+6 进入**预览**模式，你会得到两样东西：

- **带设备外框的结构预览** —— 在桌面、平板、手机三种宽度下快速看一眼大概效果。
- **真实浏览器预览** —— 软件会在你自己电脑上启动一个小服务器（地址类似 `http://127.0.0.1:8000/`），把页面交给你的日常浏览器打开，并且边改边自动刷新。

要紧的时候请用浏览器预览。 那才是真正的效果；软件内的外框只是近似示意。这一点 本手册和应用本身都会老老实实说明。

浏览器预览 / Browser preview

`../images_Images_图片/12-preview-browser.png`

Screenshot placeholder · 截图占位符

第 8 步 —— 导出

按 Ctrl+E，导出对话框会问你要哪种输出形式。第一张页面建议选**单个 .html 文件** —— 一个自包含的文件，可以邮件发送，也可以上传到任何地方。

选一个文件夹，确认，就完成了。你的页面已经变成了自己硬盘上的文件，任何浏览器都能 打开。

整个循环到这里就走完了一遍。 本手册后面所有的内容，都只是给这八步加上细节。

---

4. 界面速览

窗口分成五个区域。能叫出它们的名字，后面的一切就都好说了。



模式侧栏

最左边窄窄的一条，用来在这六件事之间切换。鼠标悬停时，状态栏会显示这个模式的 一句话说明。

模式	快捷键	用来做什么
设计	Ctrl+1	把元素拖到画布上并设置样式
积木	Ctrl+2	拼接积木，为页面添加行为
AI	Ctrl+3	让 TI-UseAgent 帮你搭建或修改
后端	Ctrl+4	把页面连接到服务器或接口
代码	Ctrl+5	查看并导出生成的源码
预览	Ctrl+6	看看浏览器里真正的样子

模式侧栏 / The mode rail

`../images_Images_图片/16-mode-rail.png`

Screenshot placeholder · 截图占位符

侧栏

侧栏会随模式变化，因为每个模式手边需要的东西不一样：

模式	侧栏显示
设计	**页面**与**图层** —— 页面列表，以及当前页面所有元素的树状结构
积木	**脚本** —— 工程里的每个脚本一条
AI	**页面** —— 助手可以看到的内容
后端	**接口端点** —— 页面可以发起的请求
代码	生成出来的**文件**
预览	预览控制项

想腾出更大的画布时，可以从**视图**菜单里把侧栏和属性面板收起来。

画布

画布是干活的地方。在设计模式下它就是你的页面，在积木模式下它是脚本区，在预览模式下 它是设备外框。

设计模式还带一小排工具：**缩放**（25% 到 200%，另有复位按钮）、可以开关的**网格** 叠加层，以及**断点**选择器 —— **桌面**、**平板**、**手机**。

断点 / Breakpoints

`../images_Images_图片/15-breakpoints.png`

Screenshot placeholder · 截图占位符

属性面板

属性面板在右侧。设计模式下，它显示当前选中元素的内容、样式和属性；其他模式下，它 显示那个模式的产出：积木模式下是生成的 JavaScript，AI 模式下是厂商配置，后端模式下 是响应结果，代码模式下是文件列表。

元素面板 / Elements panel

`../images_Images_图片/02-elements-panel.png`

Screenshot placeholder · 截图占位符

状态栏

最下面一条告诉你刚刚发生了什么、现在在哪里：一句提示或临时消息、页面数量，以及当前 页面里的元素数量。成功或失败的消息，都会先出现在这里。

---

5. 设计页面

元素面板

元素按类别分组：

类别	大致内容
布局	容器、区块、分栏、分隔线、占位块
文本	标题、段落、列表、引用、代码、标签
媒体	图片、图注、视频、音频、图标
表单	表单、输入框、多行文本、下拉框、复选框、按钮
数据	表格、术语列表、进度条
导航	导航栏、链接列表、面包屑、分页
高级	内嵌框架、画布、自定义容器、原样透传
现成版式	导航栏、主视觉、页脚、价格表、画廊、联系表单、登录表单

现成版式是一整组元素：拖一次就能得到完整的一块，然后再逐个修改里面的部分。想 让页面看起来“做完了”，这是最快的办法。

拖拽、选中、移动

- 从面板里把元素**拖**到画布上，落点会有指示线。
- **单击**元素即可选中。
- 选中后**在画布上拖动**可以移动。按住 Shift 移动会走直线。
- **吸附**帮你对齐。画布工具栏会显示吸附是否开着。

又细又深的嵌套元素有时候不好点。这时用画布右键菜单里的**选择父级**，一层一层往外 退就好。

图层与嵌套

侧栏的**图层**区把页面显示成一棵树。一个区块里有容器，容器里有标题和按钮 —— 几乎 每个页面都是这个形状。树可以帮你选中那些“点不到”的元素，也能让你看清谁可以包含谁。

两个很实用的操作：

- **包裹** —— 给选中的元素套一个新容器，这样就能把这一组当整体来设样式。
- **调整次序** —— 在同级之间把选中元素往前或往后挪。

撤销、复制、粘贴与再制

在一次会话中，撤销是没有次数限制的，而且跨编辑器通用：无论你是拖出来的改动、 在属性面板里打的字，还是让 AI 做的修改，同一个 Ctrl+Z 都能退回去。 Ctrl+Shift+Z 或 Ctrl+Y 是重做。

剪切、**复制**、**粘贴**在**编辑**菜单里。这一版有意**不**把它们绑定到 Ctrl+X / Ctrl+C / Ctrl+V：这几个键在页面上的每个输入框里都该保持原本的含义， 把它们抢走比省一次点击的代价更大。**Ctrl+D** 则在原位再制一份选中的元素。

锁定与隐藏

每个元素都有两个开关：

- **锁定** —— 元素照常显示，但不会再被误移动或误编辑。一块内容调好之后就可以锁上。
- **隐藏** —— 元素既不出现在画布上，也不会被导出。

两个开关都在属性面板里，而且都是可以撤销的普通操作。

断点：桌面、平板、手机

断点就是"允许设计发生变化的那个屏幕宽度"。本软件里有三个：

断点	可以理解成
桌面	笔记本或大显示器
平板	竖着拿的平板
手机	手机

用断点选择器切换，或者按 **Ctrl+0** 直接回到桌面。

桌面是基准。 在桌面下设定的值，会应用到所有地方，除非你另外覆盖它。

手机上的覆盖是一项刻意的例外。 假设标题在桌面下是 48 px。切到手机，选中这个 标题，把它设成 28 px。于是：

- 桌面：48 px
- 平板：**48 px** —— 继承来的，因为你从没动过平板
- 手机：28 px —— 你的覆盖值

再切回桌面，把字号改成 40 px：桌面和平板一起变成 40 px，而手机仍然是 28 px。这就是 全部的道理 —— *一个基准值，加上你确实需要的那几处例外*。想取消某个覆盖，就在那个 断点下把这个输入框清空即可。

---

6. 设置样式

选中任意元素，属性面板的三个标签页就能解决剩下的问题。

内容

文字、图片地址、按钮文字、链接目标 —— 元素"在说的事情"。初学者大部分时间都花在 这里，本来也应该是这样。

样式

分组	控制什么
快捷样式	一键预设：标题风格、卡片外观、按钮外观
文字	字体、字号、字重、对齐、行高、字间距
颜色	文字颜色、背景色、透明度
间距	元素外面留多少空、里面留多少空
圆角	角有多圆 —— 0 是方正的直角，越大越柔和
阴影	元素背后的柔和投影，卡片"浮起来"就靠它
边框	粗细、线型、颜色
布局	子元素怎么排：一个接一个自然排下去，还是自由摆放

属性

属性是元素背后的技术细节：`id`、`class` 名称、`href`、`alt` 文本，以及你自己想 附加的任何自定义数据。作为初学者，其中两个值得了解：

- 图片上的 `alt` 是屏幕阅读器念出来的文字，图片加载失败时也会显示出来。请认真 写。
- `id` 是元素的唯一名字。AI 和积木都是靠 `id` 来指认元素的，所以改名是一次 真正的改动。

不绕弯子地讲 CSS 单位

样式里的每个尺寸后面都跟着一个小单位。真正需要记住的只有五个。

单位	大白话	什么时候用
`px`	像素 —— 屏幕上的一个点	边框、细节，以及你希望"就这么大"的地方
`%`	父级尺寸的一个比例	宽度：`100%` 就是"和我的容器一样宽"
`em`	**本元素**字号的倍数	需要随文字一起变大的间距
`rem`	**整个页面**基准字号的倍数	全站统一的间距
`vw` / `vh`	视口宽度 / 高度的比例	满屏横幅：`100vh` 就是一屏高

两个能省很多麻烦的习惯：

- 间距优先用 `rem`，宽度优先用 `%`。屏幕一变，它们会自己适应。
- 手机上 `100vh` 比你能看到的高度略高一点，因为浏览器还有地址栏。如果满屏区块在 手机上感觉不对，试改成 `90vh`，然后在浏览器预览里确认一下。

---

7. 用积木编程

什么是帽子块

帽子块是顶部圆润、上面没有凹槽的那种积木，意思是*当某件事发生时*。每个脚本都 从且只从一个帽子块开始：

- **当页面加载完成** —— 页面准备好后运行一次
- **当按钮被点击** —— 某个按钮被按下时运行
- **当输入框内容改变** —— 表单字段变化时运行
- **每隔 N 秒** —— 按计时器重复运行

没有帽子块的一堆积木不成其为脚本，也不会运行。

积木怎么吸附

积木上面有凹槽、下面有对应的凸起。把一块积木拖到另一块附近，凹槽对齐时就会自动吸附。 有些积木右侧还有**插槽** —— 椭圆形的空位 —— 可以把数值类积木（数字、文本、变量） 放进去。

如果怎么都吸不上，通常是那个插槽要的是“值”，而你手里拿的是“语句”。颜色是好线索： 值和语句画法不一样。

积木类别

类别	用途
事件	帽子块 —— 什么让脚本开始运行
界面	显示与隐藏、改文字、读数值
文本	拼接文字、拆分文字、转换大小写
控制	如果、如果否则、重复、等待、停止
逻辑	比较，以及与 / 或 / 非
数学	四则运算、取整、随机数
变量	用来记住一个值的"命名盒子"
存储	让值在浏览器里跨访问保留下来
网络请求	调用接口并读取返回结果
后端	你在后端模式里定义过的端点
数据	列表与记录：新增、读取、计数
人工智能	向已配置的模型要一段简短回答
MCP **工具**	来自你已信任的 MCP 服务器的工具
定时	延时与循环计时
控制台	打印消息 —— 这是你"看见"脚本在做什么的办法

显示高级积木默认是关的。等你真的需要后面几类时再打开；第一天基本用不上。

变量

变量就是一个有名字的盒子。**设置**变量是把东西放进盒子，**改变**是在原有基础上 增减，把变量积木放出来就是在读它。名字随你起，`score`、`visitorName`、`totalPrice` 都行。

变量只在页面打开期间存在。想让一个值在刷新后还在，请改用**存储**类积木。

条件与循环

```
如果  score > 10
    显示提示  "你赢了！"
否则
    显示提示  "再试一次"
```

```
重复 5 次
    把  counter  增加 1
```

它们读起来几乎就是中文，这正是重点。把一段放进另一段里，就形成了嵌套。

用网络请求积木调用接口

接口（API） 是一个“返回数据而不是网页”的网址。套路永远是三步：

1. **发请求** —— 用网络请求积木，对某个网址执行 *GET* 或 *POST*。
2. **等回应** —— 用那个“当收到回应时”的积木，拿到结果。
3. **用结果** —— 从结果里取出一个值，放到页面上。

如果你已经在**后端**模式里定义过这个地址，**后端**类会给出一个现成的积木，就 不用重新输入网址了。

响应一次点击

最经典的第一个程序：

```
当按钮被点击
    设置  greeting  为  "你好，" + 姓名输入框的值
    设置  #output  的文字  为  greeting
```

按句子读就是：*按钮被点击时，把一句问候放进* `greeting` *变量，然后让 id 为* `output` *的元素把它显示出来。* 所谓“点击事件处理”，不过如此。

编译代码面板

积木模式会把你的积木编译成的 JavaScript 显示在**生成的 JavaScript**面板里。这不是 一个藏着掖着的转换步骤 —— 面板里显示的就是代码模式和导出真正会用到的那份输出。

面板上方有两个徽标：

- **错误** —— 有问题，代码根本生成不出来。通常是某个积木里有空输入框。填好就立刻 消失。
- **警告** —— 代码生成了，但看起来有点可疑。读一读，它们常常能替你提前挡掉一个 访客会遇到的毛病。

编译后的 JavaScript 面板 / Compiled JavaScript panel

`../images_Images_图片/05-compiled-code.png`

Screenshot placeholder · 截图占位符

你完全不必去读那段 JavaScript。它放在那儿，是为了让你**可以**读。

---

8. 连接后端

先把词说清楚

- **端点（endpoint）** 是一个你的页面可以对话的地址，比如 `https://example.com/api/contact`。它有请求方法（读用 `GET`，发用 `POST`）， 如果是 `POST`，还有请求正文 —— 也就是你要发出去的数据。
- **后端（backend）** 就是在这个地址上作答的程序。

- REST 端点是最常见的那种：一个网址加一个方法。GraphQL 端点则是一个网址， 你在正文里描述想要什么。

不做后端也能发布页面。只有当表单必须把内容发出去、或者页面必须读取会变化的数据时， 你才需要它。

定义一个端点

按 Ctrl+4 进入**后端**模式，点**添加端点**。填写：

字段	填什么
名称	给你自己看的短标签，比如 `contact`
请求方法	`GET`、`POST`、`PUT`、`PATCH` 或 `DELETE`
地址	完整网址，要带 `https://`
内容类型	现在几乎都用 `application/json`
请求头 / 查询参数	接口要求的额外请求头和网址参数
请求正文	`POST` 要发送的 JSON，可以用占位符代表表单里的值
授权	这次请求怎么证明身份：请求头、令牌，或查询参数
超时	等多久算放弃

软件会边填边校验，并告诉你还缺什么，所以你不会对着一个地址猜它到底合不合法。

后端端点 / Backend endpoints

`../images_Images_图片/09-backend-endpoints.png`

Screenshot placeholder · 截图占位符

发送测试请求

点**发送测试请求**，响应面板会显示**状态码**、**耗时**、**正文**和**响应头**。

状态码	大白话
200	一切正常，这是你要的数据
201	已创建 —— 你的 `POST` 成功了
400	服务器看不懂你发的东西
401 / 403	服务器不接受你的凭据
404	没有这个地址
500	服务器自己出错了

在软件里先测一次很关键：它能在你写任何积木之前，就把“是我的页面不对”和“是服务器 不对”分开。这是省时间最多的一步。

如果你输入了真实凭据，请放心：它们保存在工程文件**之外**，所以把 `.tcbw` 文件发给 别人，不会连密钥一起发出去。

生成配套的起始服务器

点**创建配套服务器。**软件会列出一批服务器平台，并生成一个可以立即运行的小工程， 里面已经包含你刚才定义的那些端点 —— 你刚做好的页面因此开箱即可端到端跑通。

目录里共有 **20 个目标**，从你自己部署的框架到托管平台都有：

类别	目标
Node.js	Node.js + Express、Node.js + Fastify、静态 JSON 服务器、Directus、Netlify Functions、Vercel Functions
Python	Python + FastAPI、Python + Flask
Go、Rust、PHP、Java、.NET	Go net/http、Rust + Axum、PHP + Laravel、Java + Spring Boot、.NET 最小 API
托管平台	Supabase、Firebase Functions、Cloudflare Workers、Deno Deploy、PocketBase、Appwrite、Nhost（Hasura）

其中 **7 个**有**专用生成器**，会写出完整的起始工程：**Node.js + Express**、 **Python + FastAPI**、**Cloudflare Workers**、**Supabase（Edge）** 、 **Firebase Functions**、**Go net/http** 和**静态 JSON 服务器。**选择其他目标时，软件仍然会生成一个零依赖、可运行的静态 JSON 服务器，并且用一条警告如实告知，而不是 悄悄失败 —— 这是一个诚实的兜底，而不是一个没兑现的承诺。

起始服务器目标 / Starter server targets

`../images_Images_图片/10-scaffold-targets.png`

Screenshot placeholder · 截图占位符

一个完整实例：联系表单提交到 Node/Express

1. **做表单。** 在设计模式下，把**联系表单**这块现成版式拖到画布上。姓名、邮箱、 留言框和提交按钮都已经带好了，标签和样式也都是现成的。

2. **定义端点。** 切到后端（Ctrl+4）并添加：

字段	值
名称	`contact`
请求方法	`POST`
地址	`http://127.0.0.1:3000/api/contact`
内容类型	`application/json`
请求正文	`{"name": "", "email": "", "message": ""}`

现在测一次会得到连接失败 —— 这是**正常**的，因为服务器还不存在。跳过这步，正是 大多数人误以为"表单坏了"的最常见原因。

3. **生成服务器。** 点**创建配套服务器**，选 Node.js + Express，写到工程旁边 的一个文件夹里。打开那个文件夹执行：

```
npm install
npm start
```

服务器会监听 `http://127.0.0.1:3000`，并且已经有一条 `/api/contact` 路由，另外还 有一个 `/health` 地址，用浏览器打开就能确认它还活着。

4. **再测一次。** 回到后端模式，再发一次测试请求。这次会拿到真正的响应 —— 一个 状态码和一段 JSON。你已经在碰任何积木之前，证明了整条链路是通的。

5. **接上按钮。** 进入积木模式，给提交按钮写一个脚本：

```
当提交按钮被点击
    设置 result 为 向 contact 发起 POST 请求，带上表单里的值
    如果 result 状态码 = 200
        显示提示 "谢谢，我们会尽快联系你。"
    否则
        显示提示 "出了点问题，请再试一次。"
```

6. **预览，然后松一口气。** 在浏览器里预览（Ctrl+6），把表单填好、点提交， 看着感谢提示出现，同时服务器那边打印出它刚收到的请求。

7. **导出。** 导出页面（Ctrl+E），把生成的服务器文件夹一起留着。等你把页面上 线时，记得把端点地址改成服务器真正运行的地方 —— 软件会提醒你这一点，因为 `127.0.0.1` 永远只表示"这台电脑"。

---

9. 使用 AI（TI-UseAgent）

TI-UseAgent 是软件内置的助手。它确实很有用 —— 而且它的设计前提是：**主导权在 你手里。**

选择厂商

按 Ctrl+3 进入 AI 模式。右侧面板用来配置厂商：

- **地区筛选** —— **国际**、**中国**或**本地 / 自建**。
- **厂商搜索** —— 敲几个字母就能在上百项里定位。
- **模型选择** —— 决定和哪个模型对话。
- **登录方式** —— API 密钥、OAuth，或设备码。
- **审批策略** —— 助手在多大范围内可以不经询问就动手。
- **可用工具** —— 它被允许执行的操作清单，一条不多。
- **MCP 服务器** —— 来自外部程序的额外工具（见第 10 节）。

目录里内置了 **52 家以上厂商**的预设：国际上如 OpenAI、Anthropic、Google Gemini、 Azure OpenAI、AWS Bedrock、Mistral、Cohere、xAI、Groq、Together、Fireworks、 Replicate、Hugging Face、DeepSeek、OpenRouter、Cerebras；国内如通义千问 （DashScope）、智谱 GLM、月之暗面 Kimi、百度千帆、腾讯混元、字节豆包、MiniMax、 零一万物、百川、阶跃星辰、讯飞星火、商汤、硅基流动、魔搭 ModelScope、华为 ModelArts；以及本地服务如 Ollama、LM Studio、vLLM、llama.cpp、Jan 和 text-generation-webui。

预设是**起点，不是承诺。**厂商的地址和模型名随时会变，所以每个字段都可以改 —— 预设过期了就直接修正它，不要绕过去。

AI 厂商面板 / AI provider panel

`../images_Images_图片/06-ai-provider-panel.png`

Screenshot placeholder · 截图占位符

登录

三种方式，由简到繁：

API 密钥。 最简单。在厂商网站上创建一个密钥，粘贴进来，点**保存密钥**。预设里 附带了直达厂商控制台的链接。密钥保存在**工程文件之外**，所以你可以放心地把 `.tcbw` 文件分享给别人。

OAuth 登录。 点**使用 OAuth 登录**，浏览器会打开，你在厂商自己的站点上确认， 软件随后拿到令牌。底层用的是带 PKCE 的 OAuth 2.0 —— 这是更安全的现代做法，软件 里不需要嵌入任何客户端密钥。

设备码。 适合不方便用浏览器的情况。软件给出一串短代码，你在厂商的验证页面上 输入，完成后软件自动取回令牌。

也支持直接填 Bearer 令牌。无论走哪条路，助手本身**永远拿不到你的原始凭据** —— 它只能"申请一个操作"。

审批提示：允许与不允许分别意味着什么

这里值得读两遍。

助手能做的每一件事都是**声明过的工具**，有说明、有严格限定的输入格式。清单之外的 事它做不了，也不能自己造一个新工具出来。

会改动工程的工具都要经过**审批闸门。**某个工具即将执行时，你会看到一条提示，写明 工具名、它要做什么、以及作用在哪里：

> TI-UseAgent **想修改你的页面** 请在修改发生前确认。在你允许之前，不会做任何改动。
>
> **允许 · 不允许**

- **允许** —— 改动发生。它走的是和你手动操作**完全相同**的编辑路径，所以 Ctrl+Z **一样能撤销。**
- **不允许** —— 什么都不会发生。助手会被告知你拒绝了，它可以换个方案再提。

允许和**不允许**只针对这一次操作。提示里另有"全部允许"用于一串相关改动，以及 "全部拒绝"用于你已经看够了的时候。

默认策略是**允许新增，删除前询问**：添加元素和页面不会打断你，而任何会抹掉已有 成果的操作都要先问。另外三种策略是：

策略	含义
不允许 AI 做任何修改	只读；它可以看，但不能碰
允许新增，绝不允许删除	新增自动执行；删除直接被拒绝
每次修改都询问	没有你的点击，什么都不会发生
允许新增，删除前询问	默认策略

只读工具 —— 列出页面、读取纲要、查找元素 —— 永远不会弹提示。连"看一眼"都要你 点同意，只会把你训练成不看内容就一路点下去。

审批提示 / Approval prompt

`../images_Images_图片/07-approval-prompt.png`

Screenshot placeholder · 截图占位符

工具清单

面板里按领域列出了助手可以执行的每一项操作，并标注了它是**只读**、**会修改页面**， 还是**会删除内容。**第一次登录时请读一遍。它对"这东西到底能干什么"给出的是诚实 答案。

好用的提问方式

具体一点，并且说清你**想要**什么，而不是你不想要什么：

与其说	不如说
“弄好看点”	“把标题改成深蓝色、40 px，手机上居中”
“加个表单”	“在主视觉下面加一个联系表单，有姓名、邮箱和留言”
“按钮坏了”	“「打个招呼」按钮点了没反应，让它弹出一条提示”
“有点问题”	“价格表在手机上太挤了，给它多留点空间”

助手能读取页面结构，所以你可以直接指认：*“价格表里的第二张卡片”*。它最适合一次 处理一件明确的事；遇到含糊的要求，它会说出来问清楚，而不是猜。

AI 会拒绝做的事

这些限制是刻意设计的，而且是优点：

- **默认策略下它不能未经询问就删除东西**；在两种更严格的删除策略下，它根本不能删。
- **它看不到你的 API 密钥**、OAuth 令牌或任何其他凭据。它申请的是操作，不是秘密。
- **工具清单之外的事它做不了。** 没有任意代码执行，没有命令行，没有在文件系统里 乱走。
- **除了声明过的工具允许的范围，它够不到你的工程之外。**
- **它不能保证自己的输出是对的。** 模型的输出属于不可信输入。发布出去的东西由你 负责，那一步审核是你的，不是可选项。
- **它不会帮你攻击什么。** 它是网页搭建工具，不是黑客工具。

如果某次拒绝看起来没道理，请把提示再读一遍 —— 十次里有九次，它拒绝的那个工具并不是 你以为自己在要的那个。

---

10. MCP 服务器

用大白话讲 MCP

MCP 是 *Model Context Protocol*（模型上下文协议）的缩写。它是一套通用做法， 让 AI 助手能够使用“住”在其他程序里的工具。

普通工具是内置在本软件里的 —— *添加元素*、*列出页面*。而 MCP 服务器是一个单独的 程序，它提供自己的工具：数据库读取器、设计规范查询、公司内部搜索。MCP 就是它们 共用的那个插座形状，于是助手不用为每一个都写专门代码，就能使用它们中的任何一个。

支持两种：

- **stdio** —— 运行在你自己电脑上的程序，由本软件启动并直接与之通信。
- **HTTP** —— 位于某处的服务器，通过网络访问。

信任提示，以及它为什么存在

一个 **stdio** MCP 服务器，就是一段将以**你的账户权限**在你电脑上运行的程序。同意 这件事是严肃的。

所以，在你明确把它标记为**受信任**之前，软件拒绝启动任何 MCP 服务器。在那之前， 面板上显示的是**未受信任 —— 已阻止连接**，点“连接”也不会有任何反应。这里没有 “信任全部”这种开关，因为那等于把这道防线取消掉。

信任的意思是：*我看过这是什么程序，我愿意运行它。* 请只信任你自己安装的、或者你 确实了解来源的服务器。任何你不敢双击运行的东西，就不要信任它。

> **MCP 服务器面板 / MCP servers panel**
>
> `../images_Images_图片/08-mcp-panel.png`
>
> Screenshot placeholder · 截图占位符

添加一个 stdio 服务器

1. 打开 AI 面板里的 MCP **服务器**区域，点**添加 MCP 服务器**。
2. 起一个简短的**服务器 id** —— 字母和数字，比如 `notes`。
3. 选择 **stdio** 传输方式，填入启动该程序的命令，以及它需要的参数和环境变量。
4. 保存。这一项现在显示为**未受信任**，并且没有连接任何东西。
5. 把你刚填的内容读一遍。确认你认得并且愿意运行它，才勾选**信任此服务器**。
6. 点**连接**。面板会报告这个服务器提供了多少个工具。

MCP 服务器的密钥通过**环境变量**传递，绝不放在命令行上 —— 命令行会出现在进程列表 里，环境变量块不会。

带命名空间的工具名

MCP 工具永远不会以服务器给它的名字被调用，而是被改名成：

```
mcp_<server>_<tool>
```

于是，id 为 `notes` 的服务器上那个 `search` 工具，就变成了 `mcp_notes_search`。

这不是装饰。没有命名空间的话，某个服务器可以提供一个叫 `delete_page` 的工具，悄悄 顶掉同名的内置工具 —— 而你会以为自己在批准那个无害的内置工具。命名空间让每一个 MCP 工具在工具清单和审批提示里都清清楚楚地是“外来者”。

MCP 工具走的审批闸门和其他一切完全一样。**信任一个服务器，给的是“它的工具可以被 列出来”的权利，不是“它可以动手”的权利。**

---

11. 导出与发布

四种导出格式

按 Ctrl+E，或者进入**代码**模式（Ctrl+5）点**导出**。

格式	你会得到	什么时候用
单个 .html 文件	一个自包含文件：结构、样式、脚本全在里面	想邮件发送、只上传一个文件，或者要一份"永远完整"的副本
HTML + CSS + JS	`index.html` 加上分离的样式表和脚本文件	你或别人之后要改代码；对开发者来说这是最清爽的形态
仅 JavaScript（.js）	一段在浏览器里运行时构建页面的脚本	要把页面嵌进别的东西，或喂给其他工具
完整静态站点	全部内容，外加资源、清单和说明	要正式发布一个站点，且不想有意外

文件夹里会出现什么

选完整静态站点时，大致是这样：

```
my-site/
├── index.html          首页
├── about.html          你做的每一页各一个文件
├── styles.css          样式表
├── app.js              脚本
├── manifest.json       生成了什么、什么时候生成的
├── README.txt          怎么发布
└── assets/             图片和其他文件
```

导出的结构就是普通的 HTML，样式就是普通的 CSS，行为就是普通的 JavaScript。没有 运行时，没有锁定格式，托管它们也不需要任何账号。**文件是你的。**

导出对话框 / Export dialog

`../images_Images_图片/11-export-dialog.png`

Screenshot placeholder · 截图占位符

把它放到网上

无论选哪家，步骤都一样：把导出文件夹里的内容放到某个公开位置，确保有一个页面叫 `index.html`，然后把地址告诉别人。

方式	怎么做	适合
GitHub Pages	建一个仓库，把导出的文件传上去，然后在仓库设置里开启 Pages	免费，而且你本来就熟悉 Git
Netlify **拖放**	在浏览器里把整个导出文件夹拖到它的投放页面	想要最快看到结果，不想折腾账号
任意静态主机	用 FTP、SFTP 或主机商的控制面板上传文件夹	你已经在付费的主机
自己电脑上测试	用预览服务器，或任何本地 Web 服务器	在同一个网络的另一台设备上查看

有两件事最容易踩：

- `index.html` **是正门。** 如果你的首页导出成了别的文件名，要么改回这个名字，要么 在主机的设置里把默认首页改成那个名字。
- `127.0.0.1` **只在你自己的电脑上有效。** 如果页面调用的是 `http://127.0.0.1:3000/...`，访客的浏览器会去**他们自己**的电脑上找服务器。发布 之前，请把端点改成你部署好的服务器的真实地址。

之后，你的页面就在互联网上了 —— 而它是由属于你自己的文件搭出来的。

---

12. 键盘快捷键

下表中的每一条都对照过软件自身的源码。在 macOS 上，凡写 **Ctrl** 的地方请用 **Cmd**。

快捷键	作用
Ctrl+N	新建工程
Ctrl+O	打开工程
Ctrl+S	保存
Ctrl+Shift+S	另存为
Ctrl+E	导出
Ctrl+Z	撤销
Ctrl+Shift+Z 或 Ctrl+Y	重做
Ctrl+D	再制选中的元素
Ctrl+K	呼叫 AI —— 切到 AI 模式并把光标放进输入框
Ctrl+1	设计模式
Ctrl+2	积木模式
Ctrl+3	AI 模式
Ctrl+4	后端模式
Ctrl+5	代码模式
Ctrl+6	预览模式
Ctrl+,	设置
Ctrl+0	切换到"桌面"断点
Esc	取消选中

三条能省下不少困惑的说明：

- **Delete 和 Backspace 不是全局快捷键。** 删除元素由画布处理，而画布只在没有文本 输入框获得焦点时才响应。这是刻意的：全局 Delete 会从你打字用的每一个输入框里抢走 这个键。要删除元素，请选中它，然后用编辑菜单里的**删除**，或画布上的删除按钮。
- **剪切、复制、粘贴**在**编辑**菜单里，没有绑定 Ctrl+X / Ctrl+C / Ctrl+V，这样 这几个键在文本里仍然是它们本来的意思。
- **Ctrl+0** 复位的是断点，不是缩放。缩放请用画布工具栏。

---

13. 疑难解答

现象	可能原因	怎么办
拖元素时没有反应	指针停在不是放置目标的区域，或者你拖的是分类标题而不是元素行	拖到画布上那块写着*把元素拖到这里开始*的虚线区域里放手。分类标题是展开/收起一组用的，下面那些行才是可以拖的东西
预览一片空白	没有选中页面，或者当前页面还没有任何元素，或者你正看着软件内外框而某个断点把内容藏起来了	在侧栏的页面列表里选一个页面。先放一个元素进去，才有东西可看。切回桌面断点（Ctrl+0）。还是空白就用浏览器预览，那才是准的
预览服务器启动不了	端口被占用了 —— 常见原因是上一个还开着的软件实例，或者别的程序占着	先停止预览再重新启动，软件会自动改用下一个空闲端口。把其他打开着的本软件副本关掉。如果仍然不行，先跑一次 `--doctor` 记下报告再反馈
API 密钥被拒绝	密钥错了、过期了、被吊销了、属于另一家厂商，或者该厂商要求用别的登录方式	回到厂商控制台重新复制密钥，注意前后不要带空格。确认所选的厂商预设和发密钥的是同一家。用同一个密钥在厂商自己的控制台或普通 HTTP 工具里试一次，就能定位问题在哪一端。也别忘了密钥会过期和轮换
模型列表是空的	没有选择厂商；或者该厂商的模型接口需要先通过身份验证；或者预设已经和厂商现在的接口不一致了	先选厂商，再登录 —— 有些厂商只对已认证的调用者返回模型列表。预设列表过期时可以直接手输模型名，并去厂商文档确认当前的标识
AI 一直在请求审批	审批策略设成了"每次修改都询问"，或者你的要求本身确实涉及删除	打开 AI 面板改审批策略。想让新增静默进行、只有删除才询问，就选**允许新增，删除前询问** —— 那是默认值，也是最省心的设置
导出的页面没有样式	样式表没有跟着结构一起走 —— 多文件导出时只复制了一部分，或者直接在压缩包里打开了单个文件	如果需要一个便携文件，就用**单个 .html 文件**。否则请复制**整个**导出文件夹，让 `styles.css` 和 `app.js` 待在 `index.html` 旁边。用浏览器打开导出页，在开发者控制台里看有没有"文件找不到"的报错
MCP 服务器连不上	服务器没有受信任；或者命令、地址写错了；或者那个程序根本没安装	确认这一项显示**信任此服务器**并且你已经勾选 —— 未受信任的服务器是被设计性阻断的。先在终端里单独跑一下那条命令，看能不能起来。HTTP 服务器则检查地址和是否需要令牌。仔细读面板上的报错，它会指出是哪一步失败了
整个窗口空白，或者程序一打开就退出	用户界面加载失败 —— 通常是安装不完整	重新安装，并运行 `--doctor` 把报告留下来
`--headless --export` **失败**	只写了 `--headless` 而没写 `--export`	两个必须一起给：`--headless --export <文件夹>`。单独用 `--headless` 是设计上的错误用法，会以状态码 3 退出
崩溃后未保存的内容不见了	恢复记录被丢弃了，或者这个工程从来没有保存过一次	下次启动时在欢迎界面选**恢复**。随后用 Ctrl+S 正式保存一次，以后就有真实文件可用了

现象	可能原因	怎么办
文字显示成一个个空方框	缺少覆盖该语言的字体	安装覆盖该语言的字体（中文可用微软雅黑或 Noto Sans CJK），然后重启软件

--doctor 输出 / --doctor output

`../images_Images_图片/14-troubleshooting-doctor.png`

Screenshot placeholder · 截图占位符

---

14. 术语表

术语	通俗解释
HTML	*超文本标记语言* —— 说明页面上有什么：一个标题、一段文字、一张图片。它描述结构，不描述外观
CSS	*层叠样式表* —— 说明东西长什么样：颜色、大小、间距、圆角。它描述外观
JavaScript	让页面**会做事**的语言：响应点击、获取数据、弹出提示。它描述行为
元素（element）	页面上的一个组成部分 —— 一个标题、一张图片、一个按钮。在本软件里，就是从元素面板里拖出来的东西
属性（attribute）	附加在元素上的额外信息，比如图片的 `alt` 文本、链接的 `href` 地址。写法是"名字 + 值"
积木（block）	积木模式里一块带颜色的程序片段。可以和其他积木吸附，读起来像一句话
脚本（script）	一摞积木，从一个帽子块开始。一个脚本就是一小段行为
端点（endpoint）	你的页面可以对话的一个网址，带 GET 或 POST 之类的方法
API	*应用程序接口* —— 返回数据而不是网页的服务。端点就是它的其中一扇门
API 密钥	一长串用来向服务证明"这是你"的秘密字符串。请当成密码对待：永远不要粘贴到你准备发布的页面里
OAuth	一种用别的服务来登录的办法，过程中不需要把密码交给应用。各种"用……登录"按钮就是这么工作的
MCP	*模型上下文协议* —— 一套通用的插座形状，让 AI 助手能用上由独立程序提供的工具
断点（breakpoint）	允许设计发生变化的那个屏幕宽度。本软件里有桌面、平板、手机三个
响应式（responsive）	页面能适应它所在的屏幕，而不是假定只有一种尺寸。断点就是你让页面响应式的办法

---

15. 获取帮助与反馈问题

提问之前

花三分钟准备，能让一条令人沮丧的反馈变成一次有效的修复：

1. **运行** `--doctor`，把整份报告复制下来。它一次就能回答大半关于运行环境的问题。
2. **把步骤按顺序写清楚** —— 你点了什么、先点哪个后点哪个。“它坏了”没法处理；“我把断点切到手机，改了字号，画布就白了”是可以复现的。
3. **试试浏览器预览。** 如果那边正常，问题在软件内的外框；如果那边也不正常，问题 在页面上。
4. **除了发生了什么，也说清你期望的是什么。**

到哪里提问

- **缺陷与功能建议** —— 到项目的 GitHub 页面 turtleInc/TCTools-BuildWeb 提 issue，并附上 `--doctor` 报告和你的操作步骤。
- **安全相关的问题** —— 请不要开公开 issue。按仓库 README.md 里的安全指引，改用私密漏洞报告渠道。
- **在软件内** —— **帮助**菜单里的“关于”对话框会显示版本号、署名信息，以及当前 的快捷键清单，方便你对照自己实际运行的构建。

关于这一版，两句实话

这是 **v0.1.0-rc1**，一个候选发布版本。把话说在前面：

- **软件内的预览是结构性的，不是浏览器。** 默认构建没有内置浏览器内核，所以软件内 的外框只是近似示意。通过预览服务器交给**你自己的浏览器**，才是权威效果 —— 这也是 软件同时给你两种预览的原因。
- **在 Linux 上，凭据存储是混淆，不是加密。** 密钥就以文件形式躺在数据旁边。它只能 挡住随手翻看，仅此而已。Windows 上用 DPAPI 密封并绑定你的账户；macOS 上值存在 登录钥匙串里。软件会告诉你当前属于哪一种，也从不宣称比这更多的东西。

Generated with DeepSeek V4.1 Flash on DeepSeek Harness.

本项目由 DeepSeek V4.1 Flash 在 DeepSeek Harness 上生成。

TCTools-Build_Web v0.1.0-rc1 · MIT 许可证 · 文档配图随每个发行版本发布， 位置为 `docs_Documentation_文档/images_Images_图片/`。